Em Java, a diferença fundamental na relação entre esses quatro conceitos está no **propósito de modelagem**, na **instanciação** e nas **regras de herança** (como os tipos se conectam). [1]

---

1. Resumo dos Conceitos

- **Interface:** Define um **contrato** de comportamentos (métodos) que uma estrutura deve seguir, sem guardar estado de instância (campos de objeto). [1, 2]
- **Classe Abstrata:** Define uma **base parcial** para uma hierarquia de herança. Pode ter estado (variáveis), construtores e métodos com ou sem implementação. [1, 2]
- **Classe Concreta:** É uma classe comum, totalmente implementada, que **pode ser instanciada** diretamente com `new`. [1]
- **Record:** É uma classe especial e **imutável** projetada puramente para **carregar dados** (data carrier), gerando automaticamente `equals()`, `hashCode()`, `toString()` e os métodos de leitura dos campos. [1, 2]

  Records só podem implementar interfaces, enquanto classes podem implementar 1 classe por vez e quantas interfaces quiser

---

2. A Relação entre Eles

Interfaces vs. Classes Concretas

- **Relação:** Uma classe concreta **implementa** uma ou mais interfaces usando a palavra-chave `implements`.
- **Comportamento:** A classe concreta é obrigada a fornecer o código (corpo) de todos os métodos abstratos declarados na(s) interface(s).
- **Herança Múltipla:** Uma classe concreta pode implementar **várias** interfaces ao mesmo tempo, mas só pode estender uma única classe. [1, 2]

Classes Abstratas vs. Classes Concretas

- **Relação:** Uma classe concreta **estende** (`extends`) uma classe abstrata.
- **Comportamento:** A classe abstrata serve como molde ou "meio-termo". Ela não pode ser instanciada sozinha. A classe concreta filha herda os métodos já implementados e é obrigada a implementar todos os métodos abstratos que ficaram sem corpo na classe pai.
- **Hierarquia:** Uma classe concreta só pode ter **uma** superclasse direta (seja ela concreta ou abstrata). [1, 2, 3]

Interfaces vs. Records

- **Relação:** Um `record` **não pode estender** nenhuma classe (nem abstrata nem concreta), pois ele implicitamente herda de `java.lang.Record` e é compilado como

`final`.

- **Relação com Interface:** No entanto, um `record` **pode implementar interfaces** (`implements`).
- **Comportamento:** Se a interface tiver métodos `default` ou assinaturas de métodos, o record pode adotá-los ou sobrescrevê-los, mantendo sua natureza de portador de dados imutável. [1]

Classes Abstratas vs. Records

- **Relação: Não há herança direta.** Um record **não pode** estender uma classe abstrata.
- **Por quê?** Classes abstratas podem ter estado mutável e exigem hierarquias de herança extensíveis, enquanto records são estritamente selados/finais e imutáveis por definição. [1, 2]

Tabela Comparativa de Capacidades

Característica	Interface	Classe Abstrata	Classe Concreta	Record
Pode ser instanciada?	Não	Não	Sim	Sim
Pode ter estado (campos)?	Apenas `public static final`	Sim (variáveis de instância)	Sim	Apenas os componentes declarados (privados e `final`)
Pode estender outra classe?	Não (mas estende outras interfaces)	Sim (pode `extends` outra classe)	Sim (`extends` uma classe)	Não (já herda de `Record`)

Pode implementar interfaces?	N/A	Sim (`implements`)	Sim (`implements`)	Sim (`implements`)
Objetivo principal	Definir um contrato de comportamento	Compartilhar código e base de hierarquia	Criar objetos operacionais reais	Representar dados imutáveis com sintaxe reduzida